МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Построение и анализ алгоритмов»

Тема: Поиск с возвратом

Студент гр. 4343 _____ Семенчик В.В.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург

2026

Выданное задание.

Вариант 1и. Итеративный поиск с возвратом, решение двух заданий на stepik ($2 \le N \le 30$).

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до $N-1$, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера N N . Он может получить ее, собрав из уже имеющихся обрезков(квадратов). Например, столешница размера 7×7 может быть построена из 9 обрезков.

Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные.

Размер столешницы – одно целое число N ($2 \le N \le 30$)

Выходные данные.

Одно число k – минимальное количество квадратов для покрытия столешница, потом k строк, в каждой x, y, size, разделённые пробелом, задающие координаты верхнего левого угла и длину стороны соответствующего квадрата.

Цель работы.

Изучить алгоритм поиска с возвратом.

Использованные структуры данных.

struct Square – структура для хранения координат и размера одного квадрата (обрезка).

Поля:

X, y – координаты левого верхнего угла

Size – сторона квадрата

Class Board –

Поля:

N – размер столешницы

vector<uint32_t> rows – вектор, где каждая строка представлена как 32-битная маска

vector<Square> placed_squares – множество размещённых квадратов

static uint64_t bit_prefixes[65] - статический массив предвычисленных битовых масок, где i-й элемент содержит единицы в первых i битах.

Методы:

First_empty – по переданной ссылке возвращает координаты первой пустой ячейки, если она найдена true, если вся доска заполнена false

Can_place – проверяет, возможно ли разместить квадрат заданного размера в заданных координатах

Place_square – размещает квадрат заданного размера в заданных координатах, предварительно убедившись, что это возможно

Remove_square – удаляет квадрат заданного размера в заданных координатах

Empty_cells_count – возвращает количество пустых ячеек

Стек st – хранит все состояния доски

Описание алгоритма.

В начале работы программа получает из стандартного потока ввода число N – размер столешница. Если число чётное, то решение – всегда 4 квадрата со стороной N/2. Если же N нечётное, то находится решение для наименьшего целого делителя d (начиная с 3). Если делитель не равен самому N (число составное), то найденное решение масштабируется в $k = \frac{N}{d}$ раз. Из выше сказанного следует, что далее представленный алгоритм отрабатывает только для простых нечётных чисел.

Функция find_solution.

Оптимизация 1: как первое найденное решение принимается покрытие столешницы 2N (1 квадрат размера N-1, N единичных квадратов с одной стороны, N-1 единичных квадратов с другой, 1+N+N-1=2N) квадратами, что значительно ограничивает глубину рекурсии.

Оптимизация 2: для простых нечётных чисел лучшее решение всегда содержит 1 квадрат размера N / 2 + 1, и два квадрата размера N-N/2+1. Поиск начинается не с пустой доски, а именно с такого размещения.

Работа начинается с извлечения из стека одного состояния доски. Если доска заполнена и количество квадратов на ней меньше известного лучшего решения, то текущее состояние становится лучшим известным. Если же количество квадратов не оптимально, текущая ветвь просто отбрасывается.

Оптимизация 3: если количество квадратов в текущем покрытии больше либо равно известного лучшего решения, то ветвь отбрасывается, ведь точно приведёт к неоптимальному решению

Оптимизация 4: находится число m, равное минимальному расстоянию от первой свободной точки до какой-либо из сторон. Если предположить, что все свободные ячейки получится покрыть квадратами такого размера, и всё равно количество квадратов будет больше чем в известном лучшем решении, то такая ветвь отбрасывается как неперспективная.

Далее найденный максимальный размер подгоняется в меньшую сторону так, чтобы его точно можно было разместить, и перебираются все возможные размеры от максимального найденного до 0. Для каждого из размеров новое состояние добавляется в стек.

Исследование сложности.

Временная сложность алгоритма константна для чётных чисел, для нечётных же чисел сложность $O(n^{n^2})$. Максимальная глубина рекурсии – N^2. На каждом шаге выбирается пустая клетка и возможные размеры квадрата – до N вариантов. Таким образом, размер дерева поиска = количество ветвей ^ глубина.

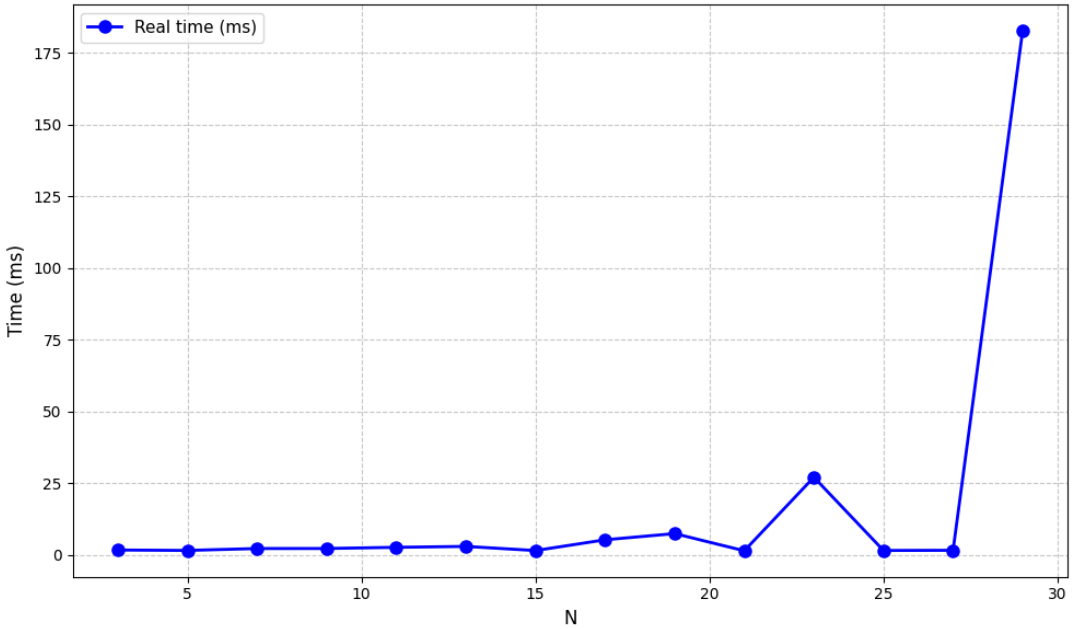

Рисунок 1. Исследование сложности алгоритма

Пространственная сложность – $O(N^4)$, ведь глубина стека ограничена N^2, а каждое состояние занимает в памяти также N^2.

Выводы.

В ходе лабораторной работы было изучен метод поиска с возвратом.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.cpp

```
#include <iostream>
#include <stack>
#include <algorithm>
#include <vector>
#include <chrono>
#include <cmath>

using namespace std;

struct Square {
    int x, y, size;
};

class Board {
public:
    int N;
    vector<uint32_t> rows;
    vector<Square> placed_squares;
    static uint64_t bit_prefixes[30];

    static void initialize_prefixes(int max_size) {
        bit_prefixes[0] = 0;
        for (int i = 1; i <= max_size + 1; i++) {
            bit_prefixes[i] = (bit_prefixes[i-1] | (1ULL << (i-1)));
        }
    }

    Board(int size) : N(size), rows(size, 0) {
        static bool initialized = false;
        if (!initialized) {
            initialize_prefixes(N);
            initialized = true;
        }
    }

    bool first_empty(int &x, int &y) {
        for (int i = 0; i < N; i++) {
            if (rows[i] != (uint64_t)((1ULL << N) - 1)) {
                for (int j = 0; j < N; j++) {
                    if (!(rows[i] & (1ULL << j))) {
                        x = i;
                        y = j;
                        return true;
                    }
                }
            }
```

```
        }
        return false;
    }

    bool can_place(int x, int y, int size) {
        if (x + size > N || y + size > N) return false;

        uint64_t mask = bit_prefixes[y + size] ^ bit_prefixes[y];

        for (int i = x; i < x + size; i++) {
            if (rows[i] & mask) return false;
        }
        return true;
    }

    void place_square(int x, int y, int size) {
        uint64_t mask = bit_prefixes[y + size] ^ bit_prefixes[y];

        for (int i = x; i < x + size; i++) {
            rows[i] |= mask;
        }

        placed_squares.push_back({x, y, size});
    }

    void remove_square(int x, int y, int size) {
        uint64_t mask = bit_prefixes[y + size] ^ bit_prefixes[y];

        for (int i = x; i < x + size; i++) {
            rows[i] &= ~mask;
        }
    }

    int empty_cells_count() {
        int count = 0;
        uint64_t full_row_mask = (1ULL << N) - 1;

        for (int i = 0; i < N; i++) {
            if (rows[i] == full_row_mask) {
                continue;
            }
            count += N - __builtin_popcountll(rows[i]);
        }
        return count;
    }
};

void print_solution(vector<Square> solution, int N) {
    cout << solution.size() << endl;
    for (auto& square : solution) {
```

```
        cout << square.x + 1 << " " << square.y + 1 << " " << square.size
<< "\n";
    }
}

uint64_t Board::bit_prefixes[30];

int smallest_divisor(int N) {
    for (int div = 3; div * div <= N; div++) {
        if (N % div == 0) return div;
    }
    return N;
}

void find_solution(int N, vector<Square> &solution) {
    stack<Board> st;
    Board board(N);

    solution.clear();
    solution.push_back({0, 0, N - 1});

    for (int i = 0; i < N; i++) solution.push_back({N - 1, i, 1});
    for (int i = 0; i < N - 1; i++) solution.push_back({i, N - 1, 1});

    int best_side = N / 2 + 1;
    board.place_square(0, 0, best_side);
    board.place_square(0, best_side, N - best_side);
    board.place_square(best_side, 0, N - best_side);
    st.push(board);

    while(!st.empty()) {
        Board current = st.top();
        st.pop();

        int x, y;
        if (!current.first_empty(x, y)) {
            if (current.placed_squares.size() < solution.size())
                solution = current.placed_squares;
            continue;
        }

        if (current.placed_squares.size() >= solution.size())  continue;

        int max_size = min({N - x, N - y});
        if (current.placed_squares.size() + current.empty_cells_count() /
(max_size * max_size) >= solution.size()) continue;


        while  (max_size  >  0  &&  !current.can_place(x,  y,  max_size))
max_size--;
```

```
        for(int size = max_size; size > 0; size--) {
            if (current.can_place(x, y, size)) {
                current.place_square(x, y, size);
                st.push(current);
                current.placed_squares.pop_back();
                current.remove_square(x, y, size);
            }
        }
    }
}
int main() {
    int N;
    cin >> N;
    auto start = std::chrono::high_resolution_clock::now();

    vector<Square> solution;
    if (N % 2 == 0) {
        int half = N / 2;
        solution.clear();
        solution.push_back({0, 0, half});
        solution.push_back({0, half, half});
        solution.push_back({half, 0, half});
        solution.push_back({half, half, half});
    } else {
        int d = smallest_divisor(N);

        find_solution(d, solution);

        if (d != N) {
            int k = N / d;

            for (auto& square : solution) {
                square.x *= k;
                square.y *= k;
                square.size *= k;
            }
        }
    }

    print_solution(solution, N);

    auto end = std::chrono::high_resolution_clock::now();
    auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(end -
start).count();
    //std::cout << ms << " ms\n";
    return 0;
}
}
```

Название файла: graphics.py

```
import subprocess
```

```
import time
import matplotlib.pyplot as plt
import numpy as np

def run_test(N):
    start = time.time()
    subprocess.run(f"echo {N} | ./main", shell=True, capture_output=True)
    end = time.time()
    return (end - start) * 1000

def main():
    sizes = list(range(3, 30, 2))
    avg_times = []

    for N in sizes:
        times = []
        for run in range(5):
            t = run_test(N)
            times.append(t)
            print(f"  N={N}, run={run+1}: {t:.2f} ms")

        avg = np.mean(times)
        avg_times.append(avg)
        print(f"N={N}: average = {avg:.2f} ms\n")

    plt.figure(figsize=(10, 6))
    plt.plot(sizes, avg_times, 'bo-', linewidth=2, markersize=8)
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.xlabel('N', fontsize=12)
    plt.ylabel('Average Time (ms)', fontsize=12)

    for i, (x, y) in enumerate(zip(sizes, avg_times)):
        plt.annotate(f'{y:.1f}', (x, y), textcoords="offset points",
                     xytext=(0,10), ha='center', fontsize=9)

    plt.tight_layout()
    plt.savefig('graph.png', dpi=100)
    #plt.show()


    with open('data.txt', 'w') as f:
        for N, t in zip(sizes, avg_times):
            f.write(f"{N} {t:.2f}\n")

if __name__ == "__main__":
    main()
```

ПРИЛОЖЕНИЕ Б

ТЕСТИРОВАНИЕ ПРОГРАММЫ

Входные данные	Выходные данные	Описание теста
3	6 1 1 2 3 1 1 3 2 1 3 3 1 1 3 1 2 3 1	Простое число
4	4 1 1 2 1 3 2 3 1 2 3 3 2	Чётное число
30	4 1 1 15 1 16 15 16 1 15 16 16 15	Чётное число, имеющее также делители кроме двойки
15	6 1 1 10 11 1 5 11 6 5 11 11 5 1 11 5 6 11 5	Составное нечётное число